LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

09

Linux Security and Hardening

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 What Is Security Hardening?
02 Principle of Least Privilege
03 SSH Security Hardening
04 Firewall Configuration
05 Patch Management
06 SELinux
07 AppArmor
08 Sudo Security
09 PAM and Password Policies
10 Securing Secrets
11 Disabling Unneeded Services
12 Auditing and Monitoring
13 Fail2ban
14 Hardening Checklist
15 Security Scenarios
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Understand how to secure Linux servers against unauthorized access and common security threats. Learn SSH hardening, firewall configuration, security updates, sudo policies, SELinux, AppArmor, auditing and practical security checks used by DevOps engineers in production environments.

1 What Is Linux Security Hardening?

Linux security hardening is the process of reducing security risks by configuring the operating system securely and disabling unnecessary access, services and features.

A newly installed Linux server may have services, accounts or configurations that are not required for its workload. Hardening helps reduce the attack surface and protect the server from unauthorized access.

Important Linux Security Practices

- Use strong authentication.
- Prefer SSH key-based authentication.
- Disable unnecessary services.
- Restrict network access using firewalls.
- Apply security updates regularly.
- Follow least-privilege access.
- Monitor system logs.
- Configure SELinux or AppArmor where appropriate.
- Audit user accounts and permissions.
- Protect sensitive configuration files and credentials.

DEVOPS NOTE

Security should be considered throughout the infrastructure lifecycle, not only after an application is deployed.

2 Understanding the Principle of Least Privilege

The principle of least privilege means users and applications should receive only the permissions required to perform their tasks.

For example, suppose you have three types of users:

User	Required access
Administrator	Manage server and services
Developer	Access application files and logs
Application user	Run the application

Do not give every user root access. Instead:

- Use individual Linux accounts.
- Grant administrative access through controlled sudo rules.
- Run applications using dedicated service accounts.
- Restrict access to sensitive files.
- Avoid unnecessary permissions such as `777`.

```
# Check current user privileges
sudo -l

# Check user and group information
id username
```

This approach limits the damage that can occur if an account or application is compromised.

3 SSH Security Hardening

SSH is commonly used to administer remote Linux servers. Because SSH provides remote access, it is an important security component.

The server-side configuration is generally stored in `/etc/ssh/sshd_config`.

Important SSH Security Settings

```
sudo nano /etc/ssh/sshd_config
```

Common hardening settings include:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers deploy ravi
```

Setting	Purpose
`PermitRootLogin no`	Prevent direct root login through SSH
`PasswordAuthentication no`	Disable password-based SSH authentication
`PubkeyAuthentication yes`	Allow SSH key authentication
`MaxAuthTries 3`	Limit authentication attempts per connection
`AllowUsers` / `AllowGroups`	Allow SSH login only for the listed users or groups

The exact configuration should account for your organization's access requirements.

> **INTERVIEW TRAP (UBUNTU)**
>
> Ubuntu also reads files from `/etc/ssh/sshd_config.d/`, and settings there take priority. On cloud images, a file such as `50-cloud-init.conf` may contain `PasswordAuthentication yes`, so password login keeps working even after you disabled it in `sshd_config`. Always check the **effective** configuration:
>
> ```
> sudo sshd -T | grep -iE "passwordauthentication|permitrootlogin"
> ```

Understanding SSH Key-Based Authentication

SSH key authentication uses a key pair:

- **Private key:** Stored securely by the client.
- **Public key:** Stored on the server.

Common locations:

```
~/.ssh/id_ed25519        (private key, on the client)
~/.ssh/id_ed25519.pub    (public key)
~/.ssh/authorized_keys   (allowed public keys, on the server)
```

```
# Generate an SSH key pair
ssh-keygen -t ed25519

# Copy the public key to a remote server
ssh-copy-id username@server-ip

# Connect using the private key
ssh -i ~/.ssh/id_ed25519 username@server-ip
```

SSH refuses keys when permissions are too open: `~/.ssh` must be `700`, and `authorized_keys` and private keys must be `600`.

Protect private keys. Never share them or upload them to public repositories.

Validate and Apply SSH Configuration

```
# Validate before applying changes
sudo sshd -t

# Reload on Ubuntu
sudo systemctl reload ssh

# Reload on RHEL-based systems
sudo systemctl reload sshd
```

On newer Ubuntu releases, SSH may be socket-activated (`ssh.socket`). If you change the SSH `Port`, run `sudo systemctl daemon-reload` and `sudo systemctl restart ssh.socket`.

> **IMPORTANT**
>
> Before disabling password authentication or changing SSH settings on a remote production server, open and test a second SSH session using the new authentication method. Keep the existing session available until access is verified. Incorrect changes can lock you out.

4 Firewall Configuration

A firewall controls incoming and outgoing network traffic based on configured rules. It helps prevent unauthorized connections to services running on a Linux server.

For example, a public web server may need:

- Port 22 for restricted SSH administration.
- Port 80 for HTTP.
- Port 443 for HTTPS.

It should not expose database ports publicly unless there is a specific, secured requirement.

UFW – Ubuntu Firewall

```
# Check firewall status
sudo ufw status

# Deny all incoming traffic by default, allow outgoing
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow SSH (or only from a trusted IP)
sudo ufw allow 22/tcp
sudo ufw allow from 203.0.113.10 to any port 22 proto tcp

# Allow HTTP and HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Enable firewall
sudo ufw enable

# Remove a rule
sudo ufw delete allow 80/tcp
```

firewalld – RHEL-Based Systems

```
# Check status
sudo firewall-cmd --state

# Allow HTTP and HTTPS permanently
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --permanent --add-service=https

# Reload rules
sudo firewall-cmd --reload

# View active configuration
sudo firewall-cmd --list-all
```

IMPORTANT FIREWALL PRINCIPLE

Allow only the ports required by your application. For example, a database server should generally accept database connections only from authorized application servers or networks.

> **AWS NOTE**
>
> When using EC2, Linux host firewalls work alongside AWS Security Groups and Network ACLs. Check all relevant network layers when troubleshooting connectivity.

5 Security Updates and Patch Management

Outdated software may contain known security vulnerabilities. Regular patching is an important part of Linux server security.

Ubuntu/Debian

```
# Update package information
sudo apt update

# Upgrade installed packages
sudo apt upgrade

# Enable automatic security updates
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades

# Check whether a reboot is required
if [ -f /var/run/reboot-required ]; then
    echo "Reboot required"
fi
```

RHEL-Based Systems

```
# Check available updates
sudo dnf check-update

# List available security updates
sudo dnf updateinfo list security

# Apply only security updates
sudo dnf upgrade --security

# Apply all updates
sudo dnf upgrade
```

The `dnf-automatic` package can apply updates automatically on RHEL-based systems.

```
# Check installed kernel version
uname -r
```

PRODUCTION RECOMMENDATION

A safe patching process generally includes:

1. Review available updates.
2. Identify security and application impact.
3. Test updates in a non-production environment.
4. Take or verify backups and recovery options.
5. Apply updates during an approved maintenance window.
6. Reboot if required.
7. Verify services and application health.

Avoid blindly upgrading production servers without testing.

6 Understanding SELinux

SELinux (Security-Enhanced Linux) is a Linux security mechanism that provides Mandatory Access Control (MAC).

Traditional Linux permissions control access using users, groups and file permissions. SELinux adds another security layer by enforcing security policies that define which processes can access which resources.

For example, even if normal Linux permissions allow access, SELinux may deny an application access to a file because the security policy does not permit it.

SELinux Modes

Mode	Description
Enforcing	Policies are actively enforced
Permissive	Policy violations are logged but not blocked
Disabled	SELinux is not active

```
# Check SELinux status
getenforce
sestatus

# Temporarily switch to permissive mode
sudo setenforce 0

# Switch back to enforcing
sudo setenforce 1
```

`setenforce` changes the mode only until the next reboot. The permanent mode is set in `/etc/selinux/config` using `SELINUX=enforcing`. Changing from disabled to enabled requires a reboot and a full filesystem relabel, which can take time.

Understanding SELinux Contexts

SELinux assigns security labels (contexts) to files and processes.

```
# View file security context
ls -Z /var/www/html

# View process security context
ps -eZ

# Restore default file contexts
sudo restorecon -Rv /var/www/html
```

Temporary vs Permanent Context Changes

Method	Behavior
`chcon`	Changes the context directly; lost on the next relabel or `restorecon`
`semanage fcontext` + **`restorecon`**	Saves the rule in the policy; permanent

```
# Permanently label a custom web directory
sudo semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
sudo restorecon -Rv /web
```

INTERVIEW TRAP: MV VS CP

`mv` keeps a file's original SELinux context, while `cp` creates a new file that inherits the context of the destination directory. If you `mv` files from your home directory into `/var/www/html`, they keep the home-directory label and the web server gets "Permission denied". Fix it with `restorecon`.

SELinux Booleans

Booleans are on/off switches that allow common behaviors without writing new policy.

```
# List web server related booleans
getsebool -a | grep httpd

# Allow the web server to make network connections (for example, as a reverse proxy)
sudo setsebool -P httpd_can_network_connect on
```

The `-P` option makes the change permanent.

SELinux Ports

```
# Show ports allowed for web servers
sudo semanage port -l | grep http_port_t

# Allow a web server to listen on port 8081
sudo semanage port -a -t http_port_t -p tcp 8081
```

The `semanage` command is provided by the `policycoreutils-python-utils` package.

Troubleshooting SELinux Denials

```
# Check recent denials (AVC messages)
sudo ausearch -m AVC -ts recent

# Explain why access was denied
sudo ausearch -m AVC -ts recent | audit2why

# Human-readable analysis with suggested fixes (setroubleshoot-server package)
sudo sealert -a /var/log/audit/audit.log
```

AVC messages indicate access-control violations.

> **IMPORTANT**
>
> Do not permanently disable SELinux simply because an application is failing. Investigate the denial and correct the policy, boolean, port or file context where appropriate.

SELinux is commonly encountered on RHEL, Rocky Linux, AlmaLinux and Fedora systems.

7 Understanding AppArmor

AppArmor is another Linux security framework that provides Mandatory Access Control. It restricts what applications can access using application-specific security profiles.

For example, an Nginx process may be allowed to:

- Read files from approved web directories.
- Access specific configuration files.
- Use required network resources.

It may be prevented from accessing unrelated sensitive directories. AppArmor is commonly enabled by default on Ubuntu.

```
# Check AppArmor status
sudo aa-status

# View AppArmor profiles
ls /etc/apparmor.d/

# Install helper tools
sudo apt install apparmor-utils

# Put a profile into complain or enforce mode
sudo aa-complain /usr/sbin/nginx
sudo aa-enforce /usr/sbin/nginx

# Find AppArmor denials
sudo journalctl -k | grep -i apparmor | grep -i denied
```

Common AppArmor Modes

Mode	Description
Enforce	Profile restrictions are applied
Complain	Violations are logged but not blocked

SELinux vs AppArmor

SELinux	AppArmor
Common on RHEL-based distributions	Common on Ubuntu
Uses security labels and policies	Uses application-specific profiles
Provides Mandatory Access Control	Provides Mandatory Access Control
Context-based policy model	Path-based profile model

Both help restrict application behavior beyond traditional Linux file permissions.

8 Managing Sudo Security

The sudo command allows authorized users to execute commands with elevated privileges. Giving unrestricted sudo access to every user is a security risk.

```
# Check sudo privileges
sudo -l

# Edit the sudo configuration safely
sudo visudo
```

A common configuration allows a group to use sudo:

```
%devops ALL=(ALL:ALL) ALL
```

This allows members of the `devops` group to execute commands through sudo.

Allow a Specific Command

```
john ALL=(root) /usr/bin/systemctl restart nginx
```

This allows user `john` to run the specified command with root privileges.

However, command restrictions must be designed carefully. Some commands can provide indirect ways to execute other commands or access sensitive files.

INTERVIEW TRAP

Allowing `sudo vim` is effectively full root access: inside Vim, the user can type `:!bash` and get a root shell. The same applies to `less`, `find`, `awk` and many other programs. To let a user edit a specific file safely, use `sudoedit`:

```
john ALL=(root) sudoedit /etc/nginx/nginx.conf
```

BEST PRACTICE

Grant only the minimum administrative privileges required, avoid `NOPASSWD` for broad rules, and review sudo permissions regularly.

9 PAM and Password Policies

PAM (Pluggable Authentication Modules) is a framework that controls how authentication works for services such as login, SSH and sudo. Administrators can change authentication rules without modifying the applications themselves.

PAM configuration is stored in `/etc/pam.d/`, with one file per service (for example `sshd` and `sudo`), plus shared files such as `common-auth` on Ubuntu and `system-auth` on RHEL.

PAM Module Types

Type	Purpose
`auth`	Verifies who the user is (password, key, MFA)
`account`	Checks whether the account is allowed (expired, locked, time restrictions)
`password`	Controls password changes and complexity
`session`	Sets up and cleans up the user session (limits, logging)

Password Complexity

Password rules are configured in `/etc/security/pwquality.conf`:

```
minlen = 12
minclass = 3
```

This requires at least 12 characters from at least 3 character classes (uppercase, lowercase, digits, symbols).

Account Lockout After Failed Logins

The `pam_faillock` module locks an account after repeated failed attempts. Settings are in `/etc/security/faillock.conf`:

```
deny = 5
unlock_time = 900
```

```
# View failed attempts for a user
sudo faillock --user john

# Unlock the user
sudo faillock --user john --reset
```

How faillock is enabled varies by distribution. Password aging is managed using `chage` (see Chapter 3).

> **WARNING**
>
> A mistake in PAM configuration can block all logins, including sudo. Keep a root session open and test changes in a second session before closing it.

10 Securing Sensitive Files and Credentials

Linux servers often contain sensitive information such as:

- Application passwords.
- Database credentials.
- API keys.
- SSH private keys.
- TLS private keys.
- Environment configuration files.

These files should not be readable by everyone.

```
# Secure an environment file
sudo chown appuser:appuser /opt/myapp/.env
sudo chmod 600 /opt/myapp/.env

# Secure SSH private keys
chmod 600 ~/.ssh/id_ed25519

# Secure the SSH directory
chmod 700 ~/.ssh

# Check file permissions
ls -l /opt/myapp/.env
```

`chmod 600` gives the owner read/write access and removes access from group and others.

Common Mistakes with Secrets

- **Passing secrets as command-line arguments:** other users can see them using `ps`.
- **Committing `.env` files to Git:** add them to `.gitignore`, and use scanners such as gitleaks to catch leaks.
- **Baking secrets into Docker images or CI/CD configuration:** anyone with access to the image or pipeline can read them.

> **DEVOPS NOTE**
>
> Avoid storing production secrets directly in Git repositories, Docker images, CI/CD configuration files or publicly accessible directories. Use a dedicated secrets management solution where possible, such as OpenBao or HashiCorp Vault, or AWS Secrets Manager and SSM Parameter Store on AWS.

11 Disable Unnecessary Services

Every unnecessary network-facing service can increase the server's attack surface.

```
# Identify running services
systemctl list-units --type=service --state=running

# Identify enabled services
systemctl list-unit-files --type=service --state=enabled

# Check which processes are listening on network ports
sudo ss -tulnp

# Stop an unnecessary service
sudo systemctl stop service-name

# Disable automatic startup
sudo systemctl disable service-name
```

IMPORTANT

Do not disable services without understanding their purpose. Some services are required for networking, storage, logging or system operation.

12 Security Auditing and Monitoring

Security hardening is not a one-time activity. Administrators should regularly inspect accounts, access, processes and logs.

Login Activity

```
# Recent login history
last

# Failed login attempts
sudo lastb | head

# Currently logged-in users
who
w
```

Authentication Logs

```
# Ubuntu and Debian
sudo grep "Failed password" /var/log/auth.log

# RHEL-based systems
sudo grep "Failed password" /var/log/secure

# Using journalctl (service is ssh on Ubuntu, sshd on RHEL)
sudo journalctl -u ssh
sudo journalctl -u sshd
```

Log locations depend on the distribution and logging configuration.

Account and File Audit Commands

```
# Accounts with UID 0 (should be only root)
awk -F: '$3 == 0 {print $1}' /etc/passwd

# Accounts with empty passwords
sudo awk -F: '$2 == "" {print $1}' /etc/shadow

# SUID files
sudo find / -xdev -perm -4000 -type f 2>/dev/null

# World-writable files
sudo find / -xdev -type f -perm -0002 2>/dev/null

# Listening ports and failed services
sudo ss -tulnp
systemctl --failed
```

auditd – Tracking Changes to Important Files

The Linux Audit system (`auditd`) records detailed security events, such as who changed a sensitive file and when.

```
# Install and start (Ubuntu; usually installed by default on RHEL)
sudo apt install auditd
sudo systemctl enable --now auditd

# Watch /etc/passwd for writes and attribute changes
sudo auditctl -w /etc/passwd -p wa -k passwd_changes

# Search recorded events by key
sudo ausearch -k passwd_changes -i

# Summary report
sudo aureport --summary
```

Rules added with `auditctl` are temporary. Save permanent rules in `/etc/audit/rules.d/` and load them using `sudo augenrules --load`.

Regular monitoring helps identify unexpected login attempts, exposed services and unusual system behavior.

13 Fail2ban – Protecting Against Repeated Login Attempts

Fail2ban is a security tool that monitors log files and can temporarily block IP addresses after repeated failed authentication attempts. It is commonly used to protect services such as SSH.

```
# Install on Ubuntu
sudo apt update
sudo apt install fail2ban

# Check service status
sudo systemctl status fail2ban

# Check active jails
sudo fail2ban-client status

# Check the SSH jail
sudo fail2ban-client status sshd

# Unban an IP address
sudo fail2ban-client set sshd unbanip 203.0.113.10
```

The exact jail name depends on the configuration.

IMPORTANT

Put your settings in `/etc/fail2ban/jail.local`, not `jail.conf`. The `jail.conf` file can be overwritten by package updates.

Fail2ban can help reduce repeated login attempts, but it does not replace SSH key authentication, firewall restrictions or proper access management. For cloud-hosted systems, also consider provider-level controls and centralized monitoring.

14 Practical DevOps Security Hardening Checklist

Suppose you have launched a new Ubuntu EC2 instance that will host a production application. A basic security checklist could look like this:

Security area	Recommended action
User access	Use individual user accounts
Root access	Avoid direct root SSH login
SSH	Prefer key-based authentication
Authentication	Disable password login when safely configured
Firewall	Allow only required ports
Software	Apply tested security updates
Services	Disable unnecessary services
Permissions	Restrict sensitive files
Application	Run under a dedicated service account
Secrets	Store credentials securely
Encryption	Encrypt data at rest (LUKS, EBS encryption) and in transit (TLS)
Auditing	Track changes to sensitive files using auditd
Monitoring	Review authentication and system logs
Recovery	Maintain backups and recovery procedures

AWS-Specific Hardening

- Restrict SSH in the Security Group to known IP addresses, or use **AWS Systems Manager Session Manager**, which needs no inbound port at all.
- Enforce **IMDSv2** so attackers cannot easily steal instance credentials through the metadata service.
- Use **IAM roles** instead of storing AWS access keys on the server.
- Encrypt EBS volumes and snapshots.

Example Basic Checks

```
# Check SSH configuration
sudo sshd -t

# Check firewall
sudo ufw status

# Check listening ports
sudo ss -tulnp

# Check failed services
systemctl --failed

# Check login history
last

# Check security framework status
sestatus
sudo aa-status
```

Not every distribution uses both SELinux and AppArmor. Check which framework is available and enabled.

Security Benchmarks and Scanning Tools

Tool / Standard	Purpose
CIS Benchmarks	Industry-standard hardening guides for each Linux distribution
Lynis	Audits a server and suggests hardening improvements
OpenSCAP	Automated compliance scanning, common on RHEL
AIDE	File integrity monitoring: detects unexpected changes to system files

```
# Run a Lynis security audit
sudo apt install lynis
sudo lynis audit system
```

15 Security Troubleshooting Scenarios

Scenario 1 Web server returns 403 after moving files (RHEL)

File permissions look correct, but Nginx or Apache cannot read files that were moved into the web directory.

```
# Check the SELinux context of the files
ls -Z /var/www/html

# Look for denials
sudo ausearch -m AVC -ts recent

# Restore the correct context
sudo restorecon -Rv /var/www/html
```

Files moved with `mv` keep their old label. For a custom web directory, add a permanent rule with `semanage fcontext` first.

Scenario 2 Nginx reverse proxy returns 502 on RHEL, but the backend works

`curl http://localhost:3000` works on the server, yet Nginx returns 502 Bad Gateway.

```
# Look for SELinux denials for nginx
sudo ausearch -m AVC -ts recent | grep nginx

# Allow the web server to make network connections
sudo setsebool -P httpd_can_network_connect on
```

By default, SELinux does not allow web server processes to connect to other network ports.

Scenario 3 Locked out after SSH hardening

After disabling password login or changing the port, nobody can connect.

- Use the cloud provider's serial console or AWS Systems Manager Session Manager if available.
- Otherwise, attach the root volume to a rescue instance and fix `sshd_config` (see Chapter 8).
- Check that the Security Group and firewall allow the new SSH port.

Prevention: run `sshd -t`, keep the existing session open and test a second session before logging out.

Scenario 4 Thousands of failed SSH login attempts in the logs

```
# Count failed attempts per IP address
sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -
```

Automated brute-force attempts are common on any server with SSH open to the internet. Disable password authentication, restrict port 22 to trusted IPs in the Security Group or firewall, and enable Fail2ban.

Scenario 5 **You suspect a server has been compromised**

1. **Do not reboot immediately.** Rebooting destroys evidence held in memory.
2. **Isolate the server:** for example, change its Security Group to block all traffic except your investigation access.
3. **Follow the incident response process** and inform the security team.
4. **Preserve evidence:** take an EBS snapshot of the disk.
5. **Investigate:**

```
# Who is logged in, and recent logins
w
last
sudo lastb | head

# Unexpected processes and listening ports
ps auxf
sudo ss -tulnp

# Persistence: cron jobs and SSH keys
sudo ls -la /etc/cron* /var/spool/cron
sudo cat /root/.ssh/authorized_keys

# Unexpected UID 0 accounts
awk -F: '$3 == 0 {print $1}' /etc/passwd

# Files changed in the last 2 days
sudo find / -xdev -type f -mtime -2 2>/dev/null | head -50
```

6. **Recover:** rebuild the server from a known-good image instead of cleaning it, rotate all credentials, keys and secrets, and patch the vulnerability that was exploited.

INTERVIEW LINE

"A compromised server cannot be trusted. Isolate it, preserve evidence, then rebuild it from a clean image and rotate all credentials."

16 Common Interview Questions

Q1. **What is Linux hardening?**

Linux hardening is the process of reducing security risks by securing configurations, restricting access, applying updates and disabling unnecessary services.

Q2. What is the principle of least privilege?

It means giving users and applications only the permissions necessary to perform their tasks.

Q3. How do you secure SSH?

Use key-based authentication, restrict root login, disable password authentication where appropriate, limit access to authorized users and monitor authentication logs.

Q4. What is SELinux?

SELinux is a Mandatory Access Control security framework that enforces policies controlling how processes access system resources.

Q5. What are the three SELinux modes?

Enforcing, Permissive and Disabled.

Q6. What is AppArmor?

AppArmor is a Mandatory Access Control framework that restricts application capabilities using security profiles.

Q7. What is the difference between SELinux and AppArmor?

Both provide application-level access restrictions. SELinux uses security labels and policy rules, while AppArmor primarily uses application-specific path-based profiles.

Q8. Why should we avoid chmod 777?

It grants read, write and execute permissions to everyone, potentially exposing files and allowing unauthorized modifications.

Q9. What is Fail2ban?

Fail2ban monitors logs for repeated failed authentication attempts and can temporarily block offending IP addresses.

Q10. How do you check open ports in Linux?

```
sudo ss -tulnp
```

Q11. How do you check failed SSH login attempts?

Check authentication logs using `journalctl`, `/var/log/auth.log` or `/var/log/secure`, depending on the distribution. `lastb` also shows failed logins.

Q12. Why should unnecessary services be disabled?

They increase the attack surface and may expose additional network services to potential attackers.

Q13. What is the difference between authentication and authorization?

Authentication verifies who a user is. Authorization determines what that user is allowed to access or perform.

Q14. How do you secure application credentials?

Restrict file permissions, avoid committing credentials to Git, use dedicated service accounts and use a secrets management solution where possible.

Q15. What is the difference between a firewall and SELinux?

A firewall controls network traffic. SELinux controls access to system resources based on security policies.

Q16. Why might a web server be denied access to files that were moved into its directory?

On SELinux systems, `mv` keeps the file's original security context. The web server is not allowed to read files with that label. Running `restorecon -Rv` on the directory applies the correct context.

Q17. What is PAM?

PAM (Pluggable Authentication Modules) is a framework that controls authentication for services such as login, SSH and sudo. It handles authentication, account checks, password rules and session setup through configurable modules in `/etc/pam.d/`.

Q18. How do you find out who modified a sensitive file such as `/etc/passwd`?

Use auditd. Add a watch rule with `auditctl -w /etc/passwd -p wa -k passwd_changes`, then search events using `ausearch -k passwd_changes`.

Q19. You set `PasswordAuthentication no`, but password login still works on Ubuntu. Why?

A file in `/etc/ssh/sshd_config.d/`, often created by cloud-init, may still set it to `yes`, and those files take priority. Check the effective setting using `sshd -T`.

Q20. Why is allowing `sudo vim` dangerous?

Vim can run shell commands, so the user can type `:!bash` and get a root shell. Use `sudoedit` to allow editing specific files safely.

Q21. How can you access an EC2 instance without opening port 22?

Use AWS Systems Manager Session Manager. It connects through the SSM agent, needs no inbound ports or SSH keys, and logs session activity.

Q22. What would you do if you suspected a server was compromised?

Avoid rebooting, isolate the server from the network, follow the incident response process, preserve evidence such as a disk snapshot, investigate logins, processes, cron jobs and SSH keys, then rebuild from a known-good image and rotate all credentials.

17 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ Linux security hardening fundamentals.
- ✓ Principle of least privilege.
- ✓ SSH key-based authentication, hardening and effective configuration.
- ✓ Firewall configuration using UFW and firewalld.
- ✓ Security patch management and automatic security updates.
- ✓ SELinux modes, contexts, booleans and denial troubleshooting.
- ✓ AppArmor and application profiles.
- ✓ Secure sudo configuration and `sudoedit`.
- ✓ PAM, password complexity and account lockout.
- ✓ Protection of application credentials and private keys.
- ✓ Disabling unnecessary services.
- ✓ Authentication logs, account audits and auditd.
- ✓ Fail2ban basics.

- ✓ AWS-specific hardening and security scanning tools.
- ✓ Responding to a suspected compromise.

PRACTICAL EXERCISES

1. Check your current user and sudo privileges.
2. Inspect SSH server configuration.
3. Validate SSH configuration using `sshd -t`.
4. Check firewall status and existing rules.
5. Identify listening network ports.
6. Review failed login attempts.
7. Check whether SELinux or AppArmor is available on your system.
8. Inspect running and enabled services.
9. Create a test sensitive file and restrict it using `chmod 600`.
10. Install Fail2ban in a test environment and inspect its status.
11. Create a dedicated application user instead of running an application as root.
12. Prepare a basic security checklist for a Linux EC2 production server.
13. Run a Lynis audit and review its suggestions.
14. Add an auditd watch on `/etc/passwd`, change a user, and find the event using `ausearch`.
15. Check the effective SSH configuration using `sudo sshd -T`.

NEXT CHAPTER

Chapter 10: Linux Shell Scripting and Automation

You will learn Bash scripting, variables, conditions, loops, functions, exit codes, error handling and practical automation scripts used in DevOps workflows.

END OF CHAPTER 9